

Spring Boot + JWT Full Guide

1. What is Spring Boot?

Spring Boot is a Java framework used to build backend applications quickly with minimal configuration.

Features:

- Auto-configuration
 - Embedded servers (Tomcat, Jetty)
 - Production-ready tools
 - Simplifies REST API development
-

2. What is JWT (JSON Web Token)?

JWT is a compact and secure way to transmit information between client and server.

Structure:

```
Header.Payload.Signature
```

Components:

- **Header:** Algorithm & token type
 - **Payload:** User data (claims)
 - **Signature:** Verifies integrity
-

3. Why Use JWT?

Traditional Authentication:

- Session-based
- Server stores user data

JWT Authentication:

- Stateless
 - Token-based
 - Scalable for microservices
-

4. Authentication Flow

1. User logs in
2. Server validates credentials

3. Server generates JWT
 4. Client stores token
 5. Client sends token with each request
 6. Server validates token
 7. Access granted
-

5. Spring Boot JWT Implementation

Dependencies:

- Spring Web
 - Spring Security
 - JWT library (jjwt)
-

5.1 Authentication Controller

```
@PostMapping("/login")
public String login(@RequestBody AuthRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            request.getUsername(),
            request.getPassword()
        )
    );

    return jwtUtil.generateToken(request.getUsername());
}
```

5.2 JWT Utility Class

```
public String generateToken(String username) {
    return Jwts.builder()
        .setSubject(username)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60))
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
        .compact();
}
```

5.3 JWT Filter

```
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(...) {
```

```
String authHeader = request.getHeader("Authorization");

if (authHeader != null && authHeader.startsWith("Bearer ")) {
    String token = authHeader.substring(7);
    String username = jwtUtil.extractUsername(token);

    // validate token
}

filterChain.doFilter(request, response);
}
```

5.4 Security Configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    protected void configure(HttpSecurity http) throws Exception {
        http.csrf().disable()
            .authorizeRequests()
            .requestMatchers("/login").permitAll()
            .anyRequest().authenticated();
    }
}
```

6. Spring Security + JWT

Spring Security handles:

- Authentication
- Authorization
- Security filters

JWT integrates using custom filters.

7. Important Concepts

Stateless Authentication

- No session stored on server

Authorization Header

```
Authorization: Bearer <token>
```

Token Expiry

- Tokens expire after a fixed time

Refresh Tokens

- Used to generate new access tokens
-

8. Advantages & Disadvantages

Advantages:

- Scalable
- Fast authentication
- Works with microservices

Disadvantages:

- Hard to revoke tokens
 - Larger size
 - Risk if token is stolen
-

9. Security Best Practices

- Always use HTTPS
 - Keep secret key secure
 - Set short expiry times
 - Use refresh tokens
 - Avoid sensitive data in payload
-

10. Real-world Use Cases

- Web applications (React + Spring Boot)
 - Mobile apps
 - Microservices architecture
-

11. JWT Methods Explained (Important)

Here are the key methods used in JWT implementation and what each one does:

11.1 generateToken()

```
public String generateToken(String username)
```

Purpose: Creates a new JWT token.

What it does:

- Sets the username as subject
 - Adds issue time
 - Adds expiration time
 - Signs the token using secret key
-

11.2 extractUsername()

```
public String extractUsername(String token)
```

Purpose: Extracts username from token.

What it does:

- Reads payload
 - Returns subject (username)
-

11.3 extractExpiration()

```
public Date extractExpiration(String token)
```

Purpose: Gets token expiry date.

What it does:

- Parses token
 - Returns expiration field
-

11.4 extractClaim()

```
public <T> T extractClaim(String token, Function<Claims, T> resolver)
```

Purpose: Generic method to extract any data.

What it does:

- Takes token
- Uses resolver function
- Returns specific claim (like username, expiry)

11.5 validateToken()

```
public Boolean validateToken(String token, UserDetails userDetails)
```

Purpose: Checks if token is valid.

What it verifies:

- Username matches
- Token is not expired

11.6 isTokenExpired()

```
private Boolean isTokenExpired(String token)
```

Purpose: Checks token expiry.

What it does:

- Compares expiry date with current time

11.7 doFilterInternal()

```
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
```

Purpose: Intercepts every request.

What it does:

- Reads Authorization header
- Extracts JWT token
- Validates token
- Sets authentication in Spring Security context

12. Summary

- generateToken() → creates token
- extractUsername() → gets user
- validateToken() → verifies token
- filter → applies security

Together, these methods form the core of JWT authentication in Spring Boot.



Spring Boot + JWT Full Guide

1. What is Spring Boot?

Spring Boot is a Java framework used to build backend applications quickly with minimal configuration.

Features:

- Auto-configuration
 - Embedded servers (Tomcat, Jetty)
 - Production-ready tools
 - Simplifies REST API development
-

2. What is JWT (JSON Web Token)?

JWT is a compact and secure way to transmit information between client and server.

Structure:

```
Header.Payload.Signature
```

Components:

- **Header:** Algorithm & token type
 - **Payload:** User data (claims)
 - **Signature:** Verifies integrity
-

3. Why Use JWT?

Traditional Authentication:

- Session-based
- Server stores user data

JWT Authentication:

- Stateless
 - Token-based
 - Scalable for microservices
-

4. Authentication Flow

1. User logs in

2. Server validates credentials
 3. Server generates JWT
 4. Client stores token
 5. Client sends token with each request
 6. Server validates token
 7. Access granted
-

5. Spring Boot JWT Implementation

Dependencies:

- Spring Web
 - Spring Security
 - JWT library (jjwt)
-

5.1 Authentication Controller

```
@PostMapping("/login")
public String login(@RequestBody AuthRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            request.getUsername(),
            request.getPassword()
        )
    );

    return jwtUtil.generateToken(request.getUsername());
}
```

5.2 JWT Utility Class

```
public String generateToken(String username) {
    return Jwts.builder()
        .setSubject(username)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60))
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
        .compact();
}
```

5.3 JWT Filter


```
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(...) {
        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            String username = jwtUtil.extractUsername(token);

            // validate token
        }

        filterChain.doFilter(request, response);
    }
}
```

5.4 Security Configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    protected void configure(HttpSecurity http) throws Exception {
        http.csrf().disable()
            .authorizeRequests()
            .requestMatchers("/login").permitAll()
            .anyRequest().authenticated();
    }
}
```

6. Spring Security + JWT

Spring Security handles:

- Authentication
- Authorization
- Security filters

JWT integrates using custom filters.

7. Important Concepts

Stateless Authentication

- No session stored on server

Authorization Header

```
Authorization: Bearer <token>
```

Token Expiry

- Tokens expire after a fixed time

Refresh Tokens

- Used to generate new access tokens

8. Advantages & Disadvantages

Advantages:

- Scalable
- Fast authentication
- Works with microservices

Disadvantages:

- Hard to revoke tokens
- Larger size
- Risk if token is stolen

9. Security Best Practices

- Always use HTTPS
- Keep secret key secure
- Set short expiry times
- Use refresh tokens
- Avoid sensitive data in payload

10. Real-world Use Cases

- Web applications (React + Spring Boot)
- Mobile apps
- Microservices architecture

11. JWT Methods Explained (Important)

Here are the key methods used in JWT implementation and what each one does:

11.1 generateToken()

```
public String generateToken(String username)
```

Purpose: Creates a new JWT token.

What it does:

- Sets the username as subject
 - Adds issue time
 - Adds expiration time
 - Signs the token using secret key
-

11.2 extractUsername()

```
public String extractUsername(String token)
```

Purpose: Extracts username from token.

What it does:

- Reads payload
 - Returns subject (username)
-

11.3 extractExpiration()

```
public Date extractExpiration(String token)
```

Purpose: Gets token expiry date.

What it does:

- Parses token
 - Returns expiration field
-

11.4 extractClaim()

```
public <T> T extractClaim(String token, Function<Claims, T> resolver)
```

Purpose: Generic method to extract any data.

What it does:

- Takes token

- Uses resolver function
 - Returns specific claim (like username, expiry)
-

11.5 validateToken()

```
public Boolean validateToken(String token, UserDetails userDetails)
```

Purpose: Checks if token is valid.

What it verifies:

- Username matches
 - Token is not expired
-

11.6 isTokenExpired()

```
private Boolean isTokenExpired(String token)
```

Purpose: Checks token expiry.

What it does:

- Compares expiry date with current time
-

11.7 doFilterInternal()

```
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
```

Purpose: Intercepts every request.

What it does:

- Reads Authorization header
 - Extracts JWT token
 - Validates token
 - Sets authentication in Spring Security context
-

12. Full JwtUtil Class (All Methods Combined)

```
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Component
public class JwtUtil {

    private final String SECRET_KEY = "your_secret_key";

    // Extract username
    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    // Extract expiration
    public Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    // Extract any claim
    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    // Extract all claims
    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .setSigningKey(SECRET_KEY)
            .parseClaimsJws(token)
            .getBody();
    }

    // Check if token expired
    private Boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    // Generate token
    public String generateToken(String username) {
        Map<String, Object> claims = new HashMap<>();
        return createToken(claims, username);
    }

    // Create token with claims
    private String createToken(Map<String, Object> claims, String subject) {
```

```
        return Jwts.builder()
            .setClaims(claims)
            .setSubject(subject)
            .setIssuedAt(new Date(System.currentTimeMillis()))
            .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 *
60)) // 1 hour
            .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
            .compact();
    }

    // Validate token
    public Boolean validateToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) &&
!isTokenExpired(token));
    }
}
```

13. Summary

- generateToken() → creates token
- extractUsername() → gets user
- validateToken() → verifies token
- filter → applies security

Together, these methods form the core of JWT authentication in Spring Boot.